

Trabajo Práctico 6: Diseño de un Sistema de Actores

Subasta Distribuida

Programación Concurrente - Primer cuatrimestre 2025
Profesores: Emilio Lopez Gabeiras y Rodrigo Pazos

1. Introducción

En este trabajo práctico se propone diseñar e implementar un **sistema de subastas** utilizando el modelo de actores con Akka. El foco principal está en las **decisiones de diseño**: qué actores existen, cómo se comunican, cómo cambian su comportamiento a lo largo del tiempo, y cómo se manejan las fallas.

2. Descripción del dominio

Una casa de subastas gestiona múltiples subastas concurrentes. El flujo general es el siguiente:

1. Un **vendedor** registra un ítem para subastar, indicando un precio base y una duración.
2. La **casa de subastas** crea una subasta para ese ítem y notifica a los postores registrados que la subasta comenzó.
3. Múltiples **postores** (bidders) envían ofertas de forma concurrente durante el período de la subasta.
4. La **subasta** solo acepta ofertas que superen la oferta actual. Si una oferta es superada, el postor anterior es notificado (“te superaron”).
5. Cuando el tiempo expira, la subasta se cierra:
 - Si hubo al menos una oferta válida, el ganador y el vendedor son notificados. Se inicia el proceso de **pago**.
 - Si no hubo ofertas, el vendedor es notificado de que el ítem no se vendió.
6. El **procesador de pagos** intenta cobrar al ganador. El cobro puede fallar (fondos insuficientes, timeout, etc.).
7. Si el pago se confirma, se notifica al vendedor. Si falla, se puede reintentar o asignar al segundo mejor postor.

3. Parte 1: Diseño

Antes de escribir código, realizar el diseño del sistema. Entregar un documento (PDF, Markdown, o README) con los siguientes puntos:

1. **Actores:** identificar todos los actores del sistema y su responsabilidad.
2. **Mensajes:** definir el protocolo de mensajes completo usando `case class` y `case object`.
3. **Diagrama de interacción:** dibujar los actores y los mensajes que se envían entre sí.
4. **Supervisión y fallas:** definir la jerarquía de supervisión y explicar cómo el sistema se recupera ante fallas.

Pistas para la investigación

Este ejercicio requiere investigar mecanismos de Akka que van más allá de lo visto en clase. Algunos conceptos a explorar:

- **Timers:** una subasta dura un tiempo fijo. Investigar `context.system.scheduler` y sus métodos `scheduleOnce` (un único envío diferido) y `scheduleWithFixedDelay` (envíos periódicos). Requieren un `ExecutionContext` (usar `import context.dispatcher`). Pensar en qué situaciones del dominio conviene cada uno.
- **Watch:** ¿cómo sabe un actor que otro actor terminó? Investigar `context.watch(actorRef)` y el mensaje `Terminated` que se recibe cuando el actor observado se detiene.
- **Lifecycle hooks:** los actores tienen métodos como `postStop()` que se ejecutan cuando el actor se detiene. Útil para liberar recursos (por ejemplo, cancelar un timer pendiente).
- **Shutdown:** investigar `system.terminate()` y `system.whenTerminated` para un cierre limpio del sistema de actores.

Además, evitar usar flags booleanas (`if (closed)`, `if (paid)`) para representar los distintos estados de un actor. Pensar en cómo `context.become` puede reemplazar esas flags, haciendo que cada estado tenga su propio comportamiento.

4. Parte 2: Implementación

Implementar el sistema diseñado en Scala con Akka. Requisitos mínimos:

1. Implementar todos los actores identificados en la Parte 1.
2. Usar `case class/case object` para todos los mensajes.
3. Usar `context.become` en al menos dos actores.

4. Implementar al menos una `supervisorStrategy`.
5. Usar `context.system.scheduler` para manejar la duración de las subastas (o un mecanismo equivalente de timeout).
6. Simular múltiples subastas y postores concurrentes.
7. Log claro del flujo. Ejemplo:

```
[AuctionHouse] New auction AUC-001: "Laptop" (base: $100, 10s)
[AuctionHouse] Notifying 3 registered bidders
[Auction-001] Bid $150 from Bidder-1 -> Accepted
[Auction-001] Bid $120 from Bidder-2 -> Rejected (current: $150)
[Auction-001] Bid $200 from Bidder-3 -> Accepted
[Auction-001] Notifying Bidder-1: outbid! (new highest: $200)
[Auction-001] Time's up! Winner: Bidder-3 at $200
[PaymentProcessor] Processing payment for AUC-001...
[PaymentProcessor] Payment confirmed for AUC-001
[Seller-1] Item "Laptop" sold for $200!
```

5. Preguntas de reflexión

Responder brevemente (2-3 oraciones cada una):

1. Si una subasta en curso tiene estado mutable (oferta actual, historial de postores) y su actor se reinicia por un crash, ¿qué pasa con ese estado? ¿Tiene sentido usar `Restart` como estrategia de supervisión para este actor? ¿Qué alternativas hay?
2. Akka usa *at-most-once* por defecto. Si una notificación de “te superaron” se pierde, el postor no sabe que su oferta ya no es la mejor. ¿Cómo podrías resolver esto a nivel de diseño?